Basi di dati - primo semestre

uniVR - Dipartimento di Informatica

Mattia Nicolis Matteo Drago

A.A. 2025-26

Indice

Introduzione **5**
Storia delle DBMS 5
Architettura di un DBMS 8

Progettazione di una base di dati **9**
Metodologie di progettazione dei dati 10
Modello entità-relazione 11
Entità 11
Identificatori delle entitità 12
Relazioni (associazioni) 13
Relazioni ternarie 14
Cardinalità delle relazioni 14
Attributi 15
Cardinalità degli attributi 16
Generalizzazione 17
Strategie di progetto 18
Analisi di qualità dello schema 18
Semplificazioni dello schema 19

Esercizi **21**
Progettazione concettuale 21
Esercizio 1 - guidato 21
Esercizio 2 22

Introduzione

Storia delle DBMS

La nascita dei sistemi per la gestione di basi di dati ha visto due momenti principali:

anni '60: sviluppo di applicazioni negli **ambienti di ricerca scientifica**

anni '70: sviluppo di applicazioni informatiche in **ambito gestionale**

Si trattava di semplici dispositivi in cui gli algoritmi di elaborazione erano semplici e grandi quantità di dati erano **condivisi** da più applicazioni.

Queste caratteristiche erano specifiche per l'ambiente in cui erano state introdotte ovvero il **sistema informativo**.

Definizione - Sistema informativo

L'insieme delle attività umane e dei dispositivi di memorizzazione ed elaborazione che organizza e gestisce l'informazione di interesse per una organizzazione di dimensioni qualsiasi.

Il sistema informativo è costituito da *dati* e *informazioni*:

- **dato**: elemento di conoscenza di base costituito da simboli che devono essere elaborati
- **informazione**: interpretazione dei dati che permette di ottenere conoscenza più o meno esatta di fatti e situazioni

Lo studio del sistema informativo avviene attraverso i **diagrammi di flusso**, questi permettono di svolgere diverse operazioni tra cui:

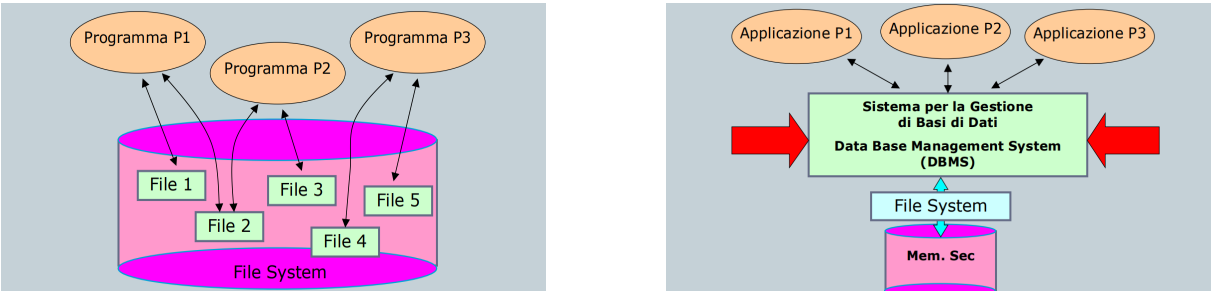
- definizione archivi dati e delle sorgenti di dati
- definizione degli utenti
- definizione di procedure e processi
- definizione dei flussi dati

Come si è arrivati allo sviluppo dei DBMS?

Negli anni '70 i programmi comunicavano direttamente con i dati contenuti nel file system.

Era un'operazione scomoda in quanto l'accesso ai dati su file era scarso (**struttura ad accesso sequenziale**), c'era ridondanza nei dati (**duplicazioni dello stesso dato su più file**), c'era inconsistenza dovuta ad aggiornamenti parziali e la progettazione dei dati era replicata per ogni programma.

Negli anni '80 la soluzione che venne proposta fu quella di inserire un **DBMS** (Data Base Management System) tra il **file system** e le **applicazioni**.



(a) Solo file system

(b) File system + base di dati

Figura 1: Prima e dopo l'innovazione

Definizione - Base di dati

Una collezione di dati utilizzati per rappresentare con tecnologia informatica le informazioni di interesse per un sistema informativo.

Definizione - DBMS

Sistema che gestisce su memoria secondaria collezioni di dati **grandi**, **condivise** e **persistenti**, assicurandone l'**affidabilità** la **privatezza** e l'**accesso efficiente**.

Questo nuovo approccio ha portato numerosi vantaggi:

- **maggiore astrazione** e più potenza espressiva per descrivere le proprietà del dato. All'interno del **DMBS** i dati vengono interpretati come oggetti, ovvero istanze di classi o righe di tabelle.

  Prima erano interpretati come blocchi o pagine di memoria secondaria (sequenze di byte)

- **operazioni di accesso ai dati più complesse** basate su un linguaggio di interrogazione (SELECT FROM WHERE) anzichè attraverso operazioni di READ/WRITE

- **migliorata l'interazione uomo-informazione:**

  – linguaggio per la definizione dei dati (Data Definition Language - DDL)

- linguaggio per l'interrogazione e aggiornamento dei dati (Data Manipulation Language – DML):
  - linguaggio di interrogazione: estrae informazioni da una base di dati (esempio: SQL, algebra relazionale)
  - linguaggio di manipolazione: popola la base di dati, modifica il suo contenuto con aggiunte, cancellazioni e variazioni sui dati (es. SQL)

Definizione - DBMS: modello dei dati

È l'insieme dei costrutti forniti dal DBMS per descrivere la struttura e le proprietà dell'informazione contenuta in una base di dati.

I costrutti permettono di definire le strutture dati che conterrano le informazioni e specificare le proprietà che dovranno soddisfare le istanze.

Nel passato esistevano modelli come quello reticolare o quello gerarchico.

Attualmente si parla di modello relazionale, ad oggetti, object-relational (SQL99 o SQL3), basato sui documenti(JSON) o NoSQL.

Definizione - Schema di una base di dati

È la descrizione della struttura e delle proprietà di una specifica base di dati fatta utilizzando i costrutti del modello dei dati (lo schema di una base di dati è invariante nel tempo).

Definizione - Istanza di una base di dati

È costituita dai valori effettivi che in un certo istante popolano le strutture dati della base di dati (l'istanza di una base di dati varia nel tempo).

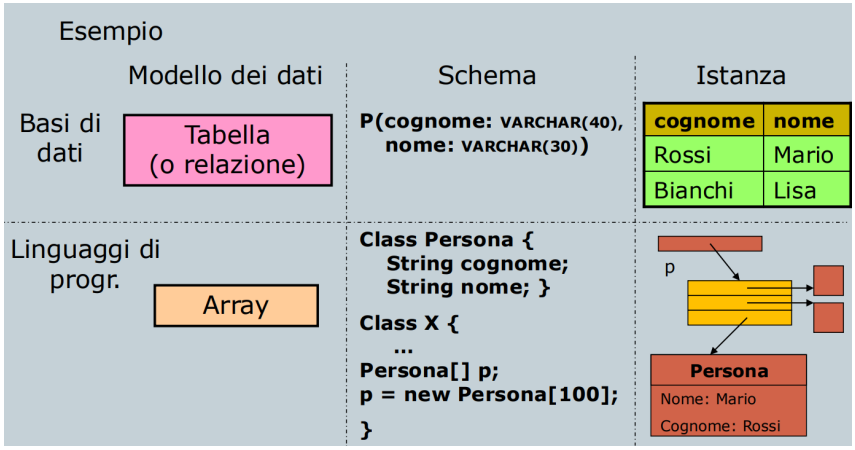


Figura 2: Distinzione tra modello, schema e istanza

Architettura di un DBMS

- **schema logico**: è la rappresentazione della struttura e delle proprietà della base di dati definita attraverso i costrutti del modello dei dati del DBMS
- **schema interno**: è la rappresentazione della base di dati per mezzo delle strutture fisiche di memorizzazione (file dati, file indice, ecc...)
- **schema esterno**: descrive una porzione dello schema logico di interesse per uno specifico utente o applicazione (attraverso viste sullo schema logico)

La caratteristica fondamentale di queste strutture è l'**indipendenza**.

indipendenza fisica: lo schema logico della base di dati è completamente indipendente dallo schema interno. Ne consegue che le variazioni delle strutture fisiche non impattano sullo schema logico e quindi sulle applicazioni.

indipendenza logica: gli schemi esterni della base di dati sono indipendendi dallo schema logico. Ne consegue che le variazioni dello schema logico (purchè non vengano rimossi dati) non impattano sugli schemi esterni e quindi sulle applicazioni (eventualmente è necessario solo ridefinire l'espressione di derivazione).

Progettazione di una base di dati

La progettazione di una base di dati costituisce solo una delle componenti del processo di sviluppo di un sistema informativo e va, quindi, inquadrata in un contesto più ampio: il **ciclo di vita** dei sistemi infoormativi.

Il *ciclo di vita* di un sistema informativo comprende:

1. **studio della fattibilità**: definisce (nel modo più preciso possibile) i *costi* delle varie alternative possibili e stabilisce le *priorità* di realizzazione delle varie componenti del sistema

2. **raccolta e analisi dei requesiti**: individua e studia le proprietà e le funzionalità del sistema, producendo una descrizioe completa, ma informale, dei dati coinvolti e delle operazione su di essi.

3. **progettazione**: si divide in

   - *progettazione dei dati*: descrive la struttura e l'organizzazione dei dati

   - *progettazione delle apllicazioni*: descrive in modo formale le caratteristiche dei programmi applicativi

   Le due attività sono complememtari e possono procedere in parallelo o in cascata.

4. **implementazione**: realizza il sistema informativo secondo la struttura e le caratteristich definite nella fase di progettazione.

   Viene costruita e popolata la base di dati e viene prodotto il codice dei programmi.

5. **validazione e collaudo**: verifica il corretto funzionamento e la qualità del sistema informativo.

   La sperimentazione deve prevedere, per quanto possibile, tutte le confizioni operative.

6. **funzionamento**: entra in funzione il sistema informativo ed esegue i compiti per i quali era stato originariamente progettato.

   Se non si verificano malfunionamenti o revisioni delle funzionalità del sistema, questa attività richiede solo operazioni di gestione e manutenzione.

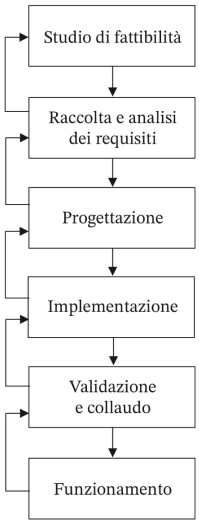


Figura 3: ciclo di vita

Metodologie di progettazione dei dati

Una **metologia di progettazione** è costituita da:

- **decomposizone** dell'intera attività di proegetto in passi sucessivi indipendeneti tra loro
- **insieme di strategie** da seguire nei vari passi e alcuni *criteri* per la scelta in caso di alternative
- **modelli di riferimento** per descrivere i dati di ingresso e uscita delle varie fasi

Le *proprietà* che una buona metologia deve avere sono:

- **generale** rispetto alle applicazioni e ai sistemi in gioco (essere inipendendete dal problema studiato e dagli strumenti utilizzati)
- **prodotto di qualità** in termini di *correttezza*, *completezza* ed *efficienza* delle risorse impiegate
- **facile da usare** nelle strategie e neo modelli di riferimento

Nell'ambito delle basi di dati, si è consolidata negli anni una metodologia di proegetto che ha dato prova di soddisfare pienamente le proprietà descritte.

Tale metodologia è articolata in tre fasi principali da effettuare a cascata:

- **progettazione concettuale**: rappresenta le specifiche informali della realtà di interesse in termini di una descrizione formale e completa, ma *indipendente* dai criteri di rappresentazione utilizzati nei sistemi di gestione di basi di dati (*schema concettuale*)
- **progettazione logica**: traduce lo schema concettuale, definito nella fase precedente, in termini del modello di rappresentazione dei dati adottato dal sistema di gestione di base di dati a disposizione (*schema logico*).

Il modello logico, ci consente di descrivere i dati secondo una rappresentazione ancora indipendente da dettagli fisici, ma concreta perchè diposnibile nei sistemi di gestione di base di dati.

- **progettazione fisica**: viene completato lo schema logico con la specifica dei parametri fisici di memorizzazione dei dati (*schema fisico*)

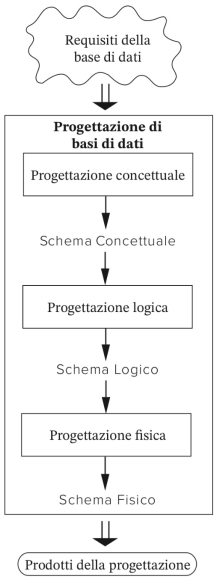


Figura 4: progettazione a cascata

Modello entità-relazione

Il **modello entità-relazione** è un *modello concettuale* di dati e fornisce una serie di strutture (*costrutti*), atte a descrivere la realtà di interesse in una maniera facile da comprendere e che prescinde dai criteri di organizzazione dei dati nei calcolatori.

Questi costrutti vengono utilizzati per definire *schemi* che descrivono l'organizzazione e la struttura delle *occorenze* dei dati, dei valori assunti dai dati al variare del tempo.

Entità

Un'**entità E** rappresenta una classe di oggetti che hanno proprietà comuni ed esistenza "autonoma" ai fini dell'applicazione di interesse (*Città*, *Dipartimento*, *impiegato*, *Acquiesto*, *Vendita* ecc.).

Un'occorenza di un'entità è un oggetto della classe che l'entità rappresenta (città di Roma, Milano e Palermo sono occorenze dell'entità *Città*).

Un'occorenza di entità ha un'esistenza indipendente dalle proprietà a esso associate.

In uno schema, ogni ha un nome che la identifica univocamente e viene rappresentata graficamente mediante un rettangono con il nome dell'intetà all'interno.

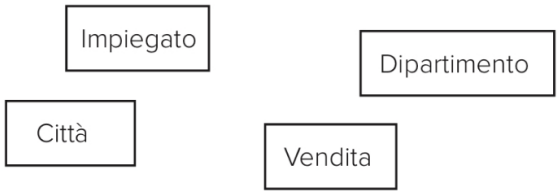


Figura 5: entità

Identificatori delle entitità

Un'**identificatore** di un'entità E è un insieme di attributi di E che consente di identificare univocamente ciascuna occorrenza di E.

In molti casi, uno o più attributi di un'entità sono sufficienti a individuare un identificatore: si parla in questo caso di **identificatore interno**.

Esercizio - Esempio di identificare interno

Un identificatore interno per l'entità *Automobile* con attributi *Targa*, *Modello*, *Colore* e *Anno di immatricolazione* è costituito dall'attributo *Targa*, in quanto ogni automobile ha una targa univoca e non posso esistere due automobili con la stessa targa.

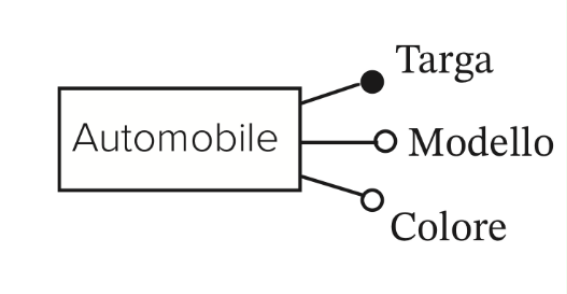


Figura 6: identificatore interno

Nei casi in cui l'identificazione di un'entità è ottenuta utilizzando attributi di altre entità, si parla di **identificatore esterno**.

Esercizio - Esempio di identificatore esterno

Nell'entità *Studente*, sembrerebbe a prima vista che l'attributo *Matricola* possa essere un identificatore interno, in quanto ogni studente ha una matricola univoca. Tuttavia, se si considera che la matricola viene assegnata da un'università, si può notare che due studenti iscritti a due università diverse potrebbero avere la stessa matricola.
In questo caso, un identificatore esterno per l'entità *Studente* potrebbe essere costituito dagli attributi *Matricola* e *Università*, in quanto la combinazione di questi due attributi consente di identificare univocamente ciascuno studente.

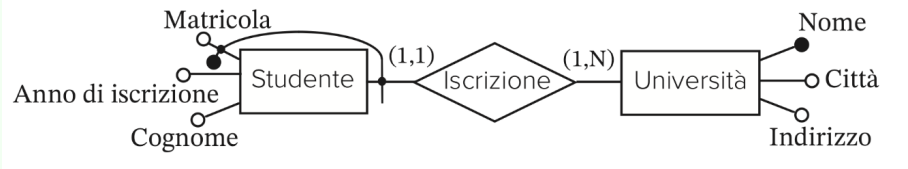


Figura 7: identificatore esterno

Relazioni (associazioni)

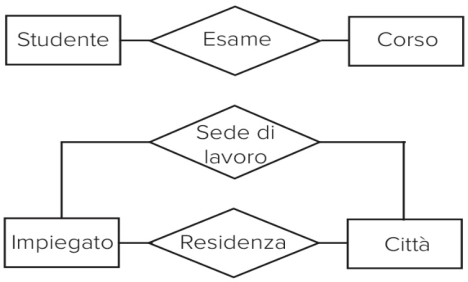
Una **relazione R** rappresenta un legame logico tra due o più entità (*Residenza* è una relazione che può sussistere tra le entità *Città* e *Impiegato*).

Un'occorenza di relazione è un'ennupla costituita da occorenze di entità, una per ciascuna delle entità coinvolte.

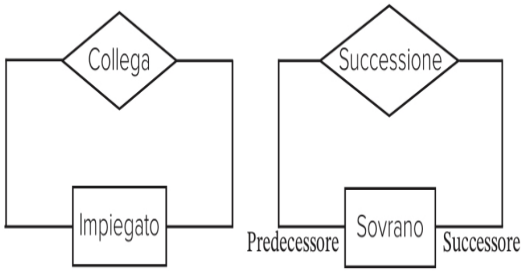
In uno schema E-R, ogni relazione ha un nome che identifica univocamente e viene rappresentata graficamente mediante un rombo con il nome delle realzione al'interno e da linee che connettono la relazione con ciascuna delle sue componenti.

L'insieme delle occorrenze di una relazione del modello E-R è, una relazione matematica tra le occorenze delle entità coinvolte, ossia, un sottoinsieme del loro prodotto cartesiano.

Questo significa che tra le occorenze di una relazione del modello E-E non ci possono essere ennuple ripeute.



(a) relazioni tra entità



(b) relazione ricorsiva

E' anche possibile avere *relazioni ricorsive*: relazioni tra un'entità e se stessa.

E' possibile, infine, avere *relazioni n-arie* che coinvolgono più di due entità.

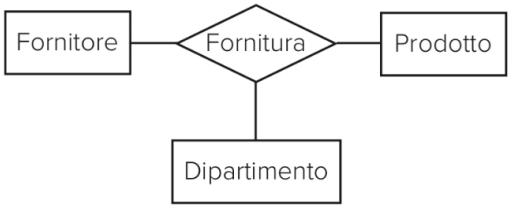


Figura 9: relazione ternaria

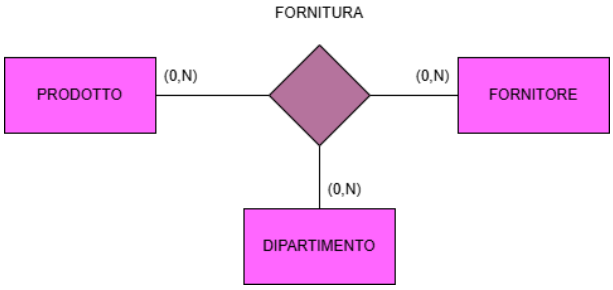
Relazioni ternarie

La relazione ternaria è un particolare tipo di relazione n-aria che viene utilizzata quando:

- un'istanza del concetto per esistere nel sistema informativo (e quindi nella base di dati) richiede sempre la presenza di tre istanze di entità (una per ogni entità coinvolta nella relazione).
- Non esistono eccezioni al punto precedente.
- Non esistono situazioni nelle quali la terna di entità debba essere rappresentata più volte.

Esempio di uso corretto:

Vogliamo rappresentare il fatto che un prodotto venga fornito ad un dipartimento da un fornitore. Ogni dipartimento può scegliere quali prodotti ordinare dai fornitori selezionando un sottoinsieme dei profotto forniti dal fornitore. Ogni fornitore vende a più dipartimenti prodotto diversi. Ogni prodotto viene venduto da più fornitori.



dove le istanze hanno le sequenti caratteristiche:

I(FORNITORE) = {f1,f2,f3}

I(PRODOTTO) = {p1, p2, p3}

I(DIPARTIMENTO) = {d1, d2}

I(FORNITURA) ={(f1,d1,p1), (f1,d2,p3), (f3,d1,p3), (f3,d2,p2)}

Cardinalità delle relazioni

La **cardinalità** viene specificata per ciascuna partecipazione di entità a una relazione ed esprime il numero (minimo, massimo) di occorenze di un'entità che possono essere associate ad una occorrenza di un'altra entità attraverso la relazione.

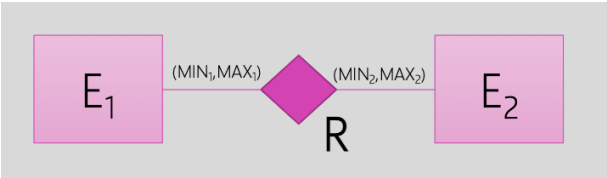


Figura 10: cardinalità delle relazioni

In linea di principio la cardinalità può essere qualsiasi coppia di numeri naturali (incluso 0 e ∞), con l'unico vincolo che la cardinalità minima deve essere minore o uguale della cardinalità massima.

In realtà, nella maggior parte dei casi, le *cardinalità minime* sono 0 o 1 e le *cardinalità massime* sono 1 o N.

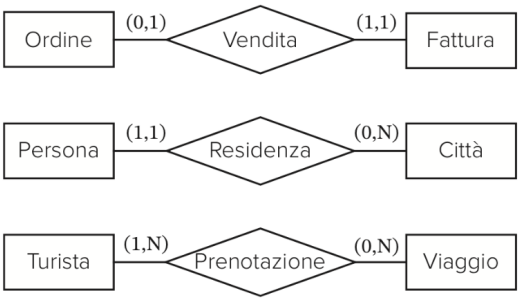


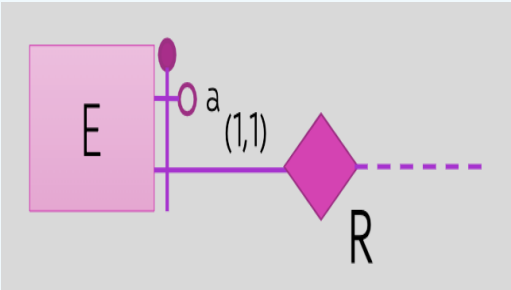
Figura 11: esempi di cardinalità

cardinalità minima 0 o 1 : significa che la partecipazione di un'entità ad una relazione può essere *facoltativa* (0) o *obbligatoria* (1)

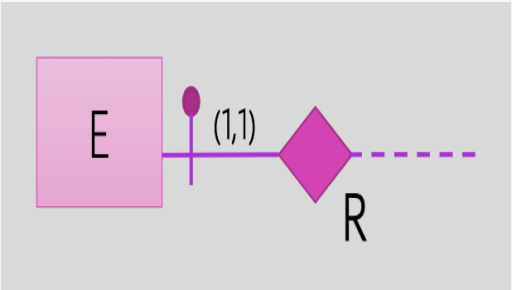
cardinalità massima 1 o N : significa che la partecipazione di un'entità ad una relazione può essere *singola* (1) o *multipla* (N)

Definizione - N.B.

Una relazione R può partecipare ad un identificatore esterno di un'entità E solo se la tale entità partecipa alla relazione con vincolo di cardinalità (1,1) o R è la sola proprietà che partecipa all'identificatore esterno di E.



(a) caso 1



(b) caso 2

Attributi

Un **attributo** descrive le proprietà elementari di un'entità (o relazione) (*Cognome*, *Stipendio* ed *Età* sono possibili attributi dell'enità *Impiegato*).

Un attributo associa a ciascuna occorrenza di entità (o di relazione) un valore appartenente a un insieme (*dominio*), che contiene i valori ammissibili per l'attributo.

I domini non vengono riportati nello schema, ma sono generalmente nella documentazione associata.

Può risultare comodo raggruppare attributi della medesima entità (o relazione) che presentano affinità nel loro significto o uso (*attributo composto*), ad esempio *Via*, *Numero civico* e *CAP* possono formare l'attributo composto *Indirrizzo* dell'entità *Persona*.

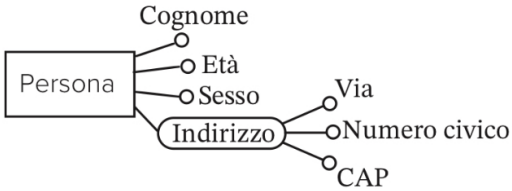


Figura 12: attributo composto

Un esempio completo di **modello E-R**, può essere il seguente:

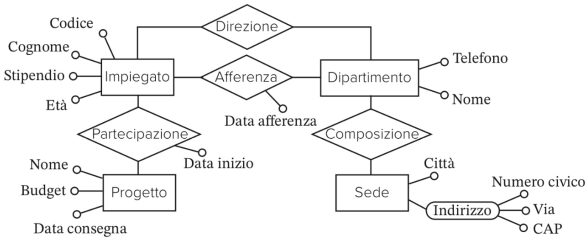


Figura 13: modello E-R completo

Cardinalità degli attributi

Possono essere specificate specificate per gli attributi di un'entità (o relazione) le **cardinalità** che esprimono il numero (minimo, massimo) di valori che l'attributo può assumere per ciascuna occorrenza dell'entità (o relazione).

Nella maggior parte dei casi, le cardinalità di un attributo è pari a (1,1) e viene omessa nella rappresentazione grafica: in questi casi l'attributo rappresenta sostenzialmente una funzione che associa a ogni occorenza dell'entità (o relazione) un unico valore del dominio dell'attributo.

Il valore per un certo attributo può essere *nullo* (cardinalità minima 0) o possono essere associati più valori (cardinalità massima N): nel primo caso si dice che la partecipazione dell'attributo è **opzionale**, nel secondo caso si dice che la partecipazione è **obblogatoria**.

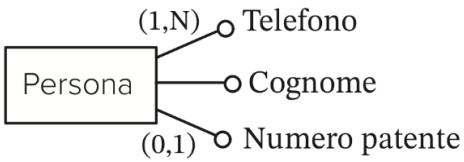


Figura 14: cardinalità degli attributi

Un attributo con cardinalità massima N viene detto **attributo multivalore** e devono essere utilizzati con cautela, perchè rappresentano situazioni che possono essere modellate,

in alcune occasioni, in modo più appropriato mediante l'introduzione di nuove entità e relazioni.

Generalizzazione

La **generalizzazione** è un meccanismo di astrazione che consente di raggruppare entità che condividono alcune proprietà comuni in una entità più generale.

Rappresentano legami logici tra un'entità E (entità genitore) e una o più entità *E1, E2, ..., En* (entità figlie), dove ciascuna entità figlia E_i è una specializzazione dell'entità genitore E.

Esercizio - Esempio di generelizzazione

L'entità *Persona* è una generelizzazione delle entità *Uomo* e *Donna*, in quanto entrambe le entità figlie condividono alcune proprietà comuni con l'entità genitore, come il nome, il cognome e la data di nascita.

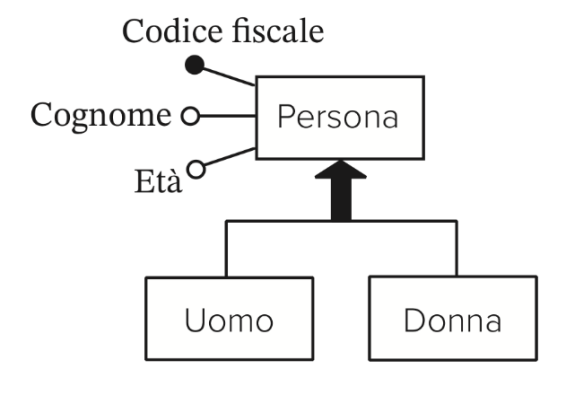


Figura 15: generalizzazione

Le generalizzazioni vengono rappresentate graficamente mediante delle frecce che collegano l'entità genitore alle entità figlie.

Le generalizzazioni possono essere di due tipi:

- **totale**: ogni occorrenza dell'entità genitore deve essere anche un'occorrenza di una delle entità figlie (altrimenti è **parziale**)
- **esclusiva**: ogni occorrenza dell'entità genitore può essere anche un'occorrenza di una sola delle entità figlie (altrimenti è **sovrapposta**)

La generelizzazione tra *Persona*, *Uomo* e *Donna* è totale ed esclusiva, in quanto ogni persona deve essere o un uomo o una donna e non può essere entrambe le cose.

Strategie di progetto

Lo sviluppo di uno schema concettuale può essere visto come un processo di ingegnerizzazione. È possibile applicare due strategie classice:

- **TOP-DOWN**:
  - Fase 1: considerare le specifiche GLOBALMENTE e produrre uno schema iniziale completo ma con POCHI CONCETTI molto ASTRATTI.
  - Fase 2: RAFFINARE i concetti astratti fino ad arrivare allo schema concettuale COMPLETO in ogni DETTAGLLIO
- **BOTTOM-UP**:
  - Fase 1: DECOMPORRE le specifiche iniziali in PARTI ELEMENTARI (frasi che descrivono lo stesso concetto)
  - Fase 2: GENERARE gli SCHEMI per tutte le parti elementari individuate.
  - Fase 3: FONDERE gli schemi (introducendo altri costrutti del modello E-R) in modo da INTEGRARE tutti gli schemi componenti e generare lo SCHEMA FINALE.
- INSIDE-OUT:
  - Fase 1: INDIVIDUARE nelle specifiche alcuni CONCETTI IMPORTANTI (detti concetti GUIDA)
  - Fase 2: GENERARE gli SCHEMI per i concetti GUIDA
  - Fase 3: FONDERE gli schemi precedenti e generare lo SCHEMA FINALE

Analisi di qualità dello schema

Verifica di correttezza: uno schema E-R è corretto se utilizza propriamente i construtti del modello concettuale.
Possibili errori: sintattici, semantici(errori d'uso).

Verifica di completezza: uno schema E-R è completo se rappresenta tutte le informazioni di interesse e se tutte le operazioni descritte nelle specifiche possono essere eseguite a partire dai dato dello schema.

Verifica di minimalità: quando tutti i concetti descritti nei requisiti sono rappresentati nello schema UNA VOLTA SOLA

Verifica di leggibilità: quando rappresenta i requisiti in maniera naturale facilmente comprensibile. Attenzione ai nomi (di entità, relazioni e attributi), e alle rappresentazioni contorte.

Uno schema concettuale **non minimale** contiene ridondanze.

Casi di ridondanza:

- **Ereditarietà delle Relazioni**

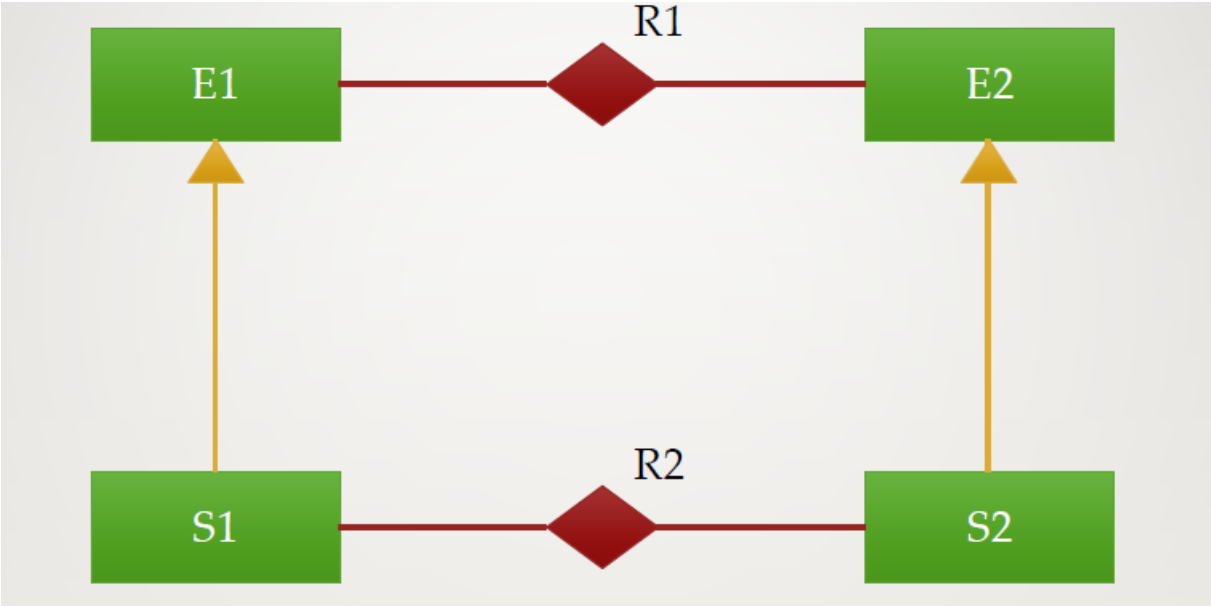


Figura 16: esempio ereditarietà

È probabile, ma non certo, che R_2 sia ridondante. Va quindi verificato se R_2 rappresenta lo stesso legame logico di R_1, e se è cosi, R_2 va eliminata.

- **Cicli di Relazioni**

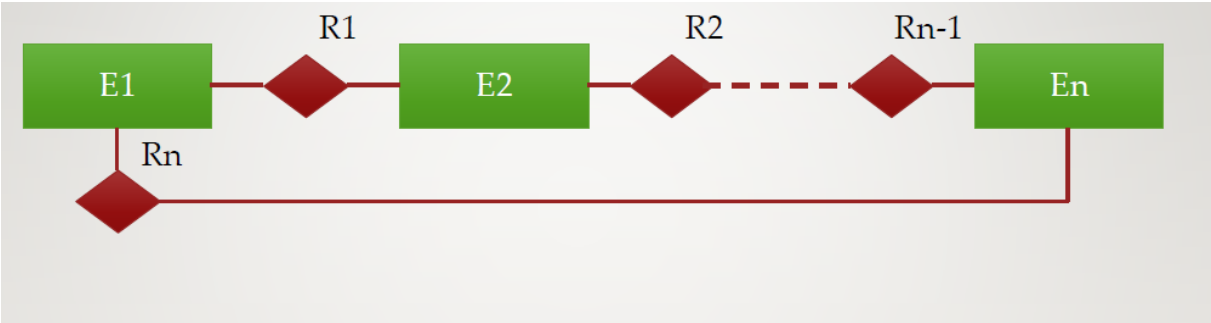


Figura 17: esempio cicli di relazioni

È possibile che una delle relazioni R_i sia ridondante. Va quindi verificato se R_i si possa ottenere dalla composizione delle altre relazioni e se così è R_i va eliminata in quanto effettivamente ridondante.

Semplificazioni dello schema

Non sempre le relazioni ternarie sono facili da leggere ed interpretare, quindi solitamente la loro eliminazione può comportare una migliore leggibilità dello schema.

Operazioni possibili:

- Trasformazione di una relazione ternaria in una entità(sempre applicabile)

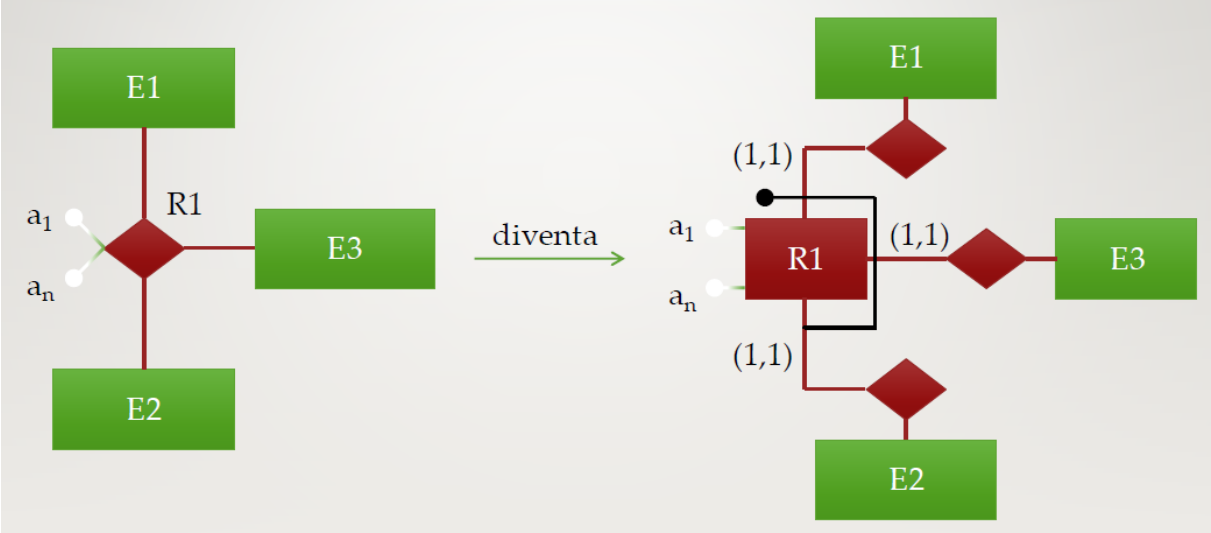


Figura 18: Trasformazione relazione ternaria

- riduzione di una relazione ternaria a due relazioni binarie (PRECONDIZIONE: una delle entità partecipa alla relazione ternaria con cardinalità(1,1))

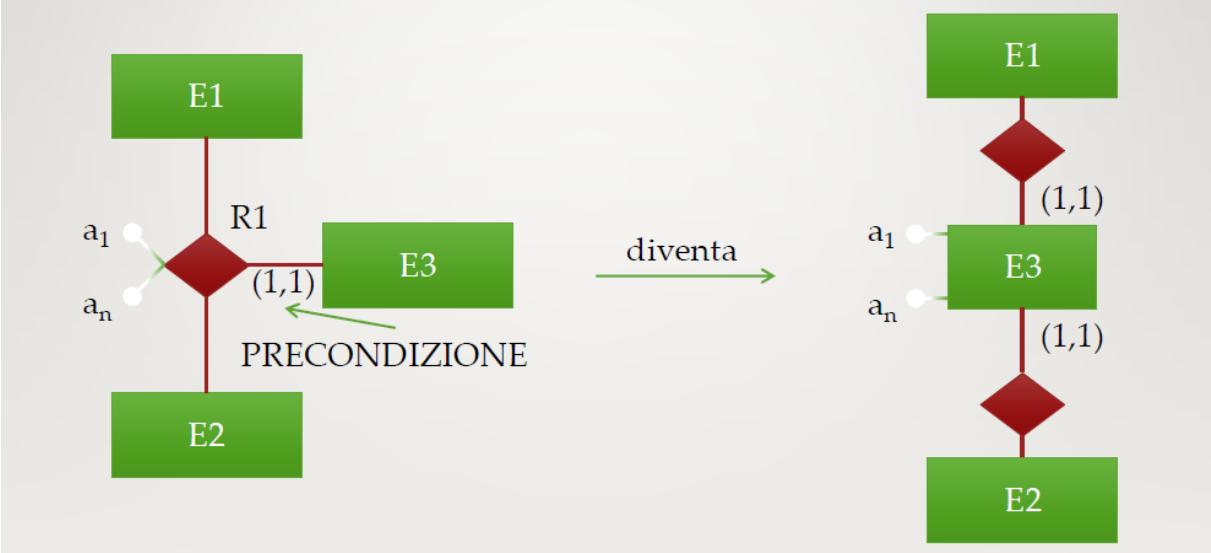


Figura 19: Eliminazione della relazione ternaria

Esercizi

Progettazione concettuale

Esercizio 1 - guidato

Esercizio - Progettazione concettuale

Progettare una base di dati che contenga le informazioni che descrivono la carriera di uno studente del corso di laurea triennale in informatica dal momento dell'immatricolazione fino alla laurea. In particolare, il sistema registra tutti gli esami sostenuti dagli studenti durante la loro carriera.
Per ogni studente si memorizzano: il nome, il cognome, la data di nascita, la matricola (univoca), la data di immatricolazione e la data di laurea. Per ogni insegnamento erogato si memorizzano: la denominazione, l'anno accademico di erogazione, l'anno di corso (I, II o III) e il docente.
Si vuole tener traccia nella base di dati per ogni studente:

- degli insegnamenti frequentati indicando la percentuale di presenza (> 0) di uno studente alle lezioni e
- degli esami sostenuti indicando la data dell'esame e il voto finale.

Si vuole inoltre poter calcolare quanti studenti hanno frequentato un insegnamento.

Soluzione iniziale fornita dal prof (errata):

Questa soluzione presenta **diverse problematiche:**

1. L'utente è legato all'esame, infatti un utente esiste solo se esiste anche un'esame. Per questo motivo sarebbe consono creare un'entità utente contenente nome, cognome, matricola...

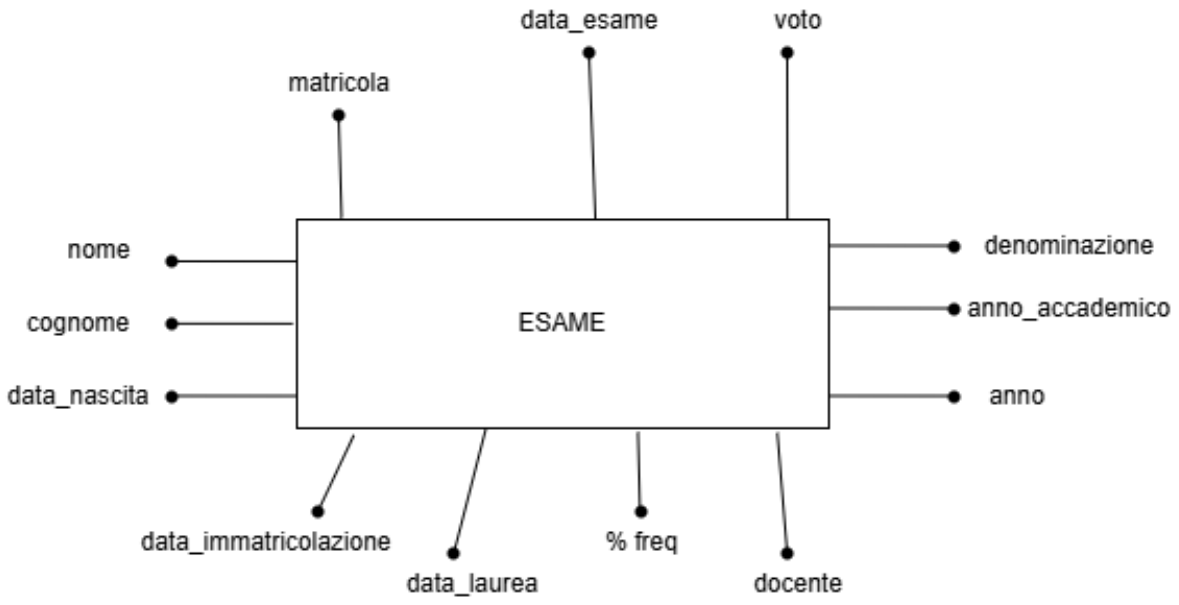


Figura 20: esercizio

2. Cosa identifica nello specifica l'entità esame? Un appello? Un esame completo...?
3. Dentro all'entitità dovrebbero esserci oggetti appartenneti ad un'unica classe (oggetti simili)

Tutto questo porta a **ridondanza e ritardo nella memorizzazione dei dati.**

Una possibile soluzione dell'esercizio è questa.

Come si può vedere, sono state create delle entità per studente e insegnamento. L'esame è diventato una relazione che lega lo studente all'insegnamento, cosi come lo è diventata la frequenza.

Esercizio 2

Esercizio - Base di dati di una banca

Si vuole progettare la basi di dati di una banca. La banca è suddivisa in filiali e di ogni filiale si conosce il nome, il codice numerico (univoco), l'indirizzo e il numero di clienti.
I clienti della banca sono memorizzati nella base di dati. Per ogni cliente sono memorizzati: il nome, il cognome, il codice fiscale, la data di nascita e la professione.
La base di dati contiene inoltre i dati sui conti correnti aperti presso le

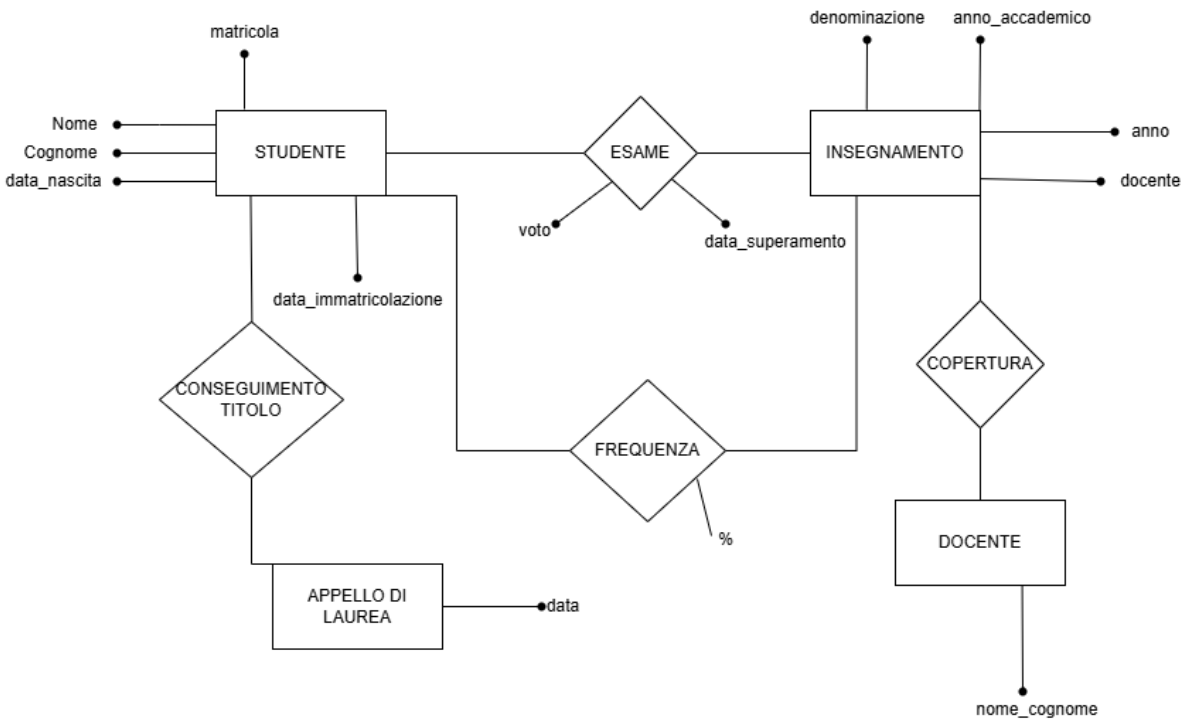
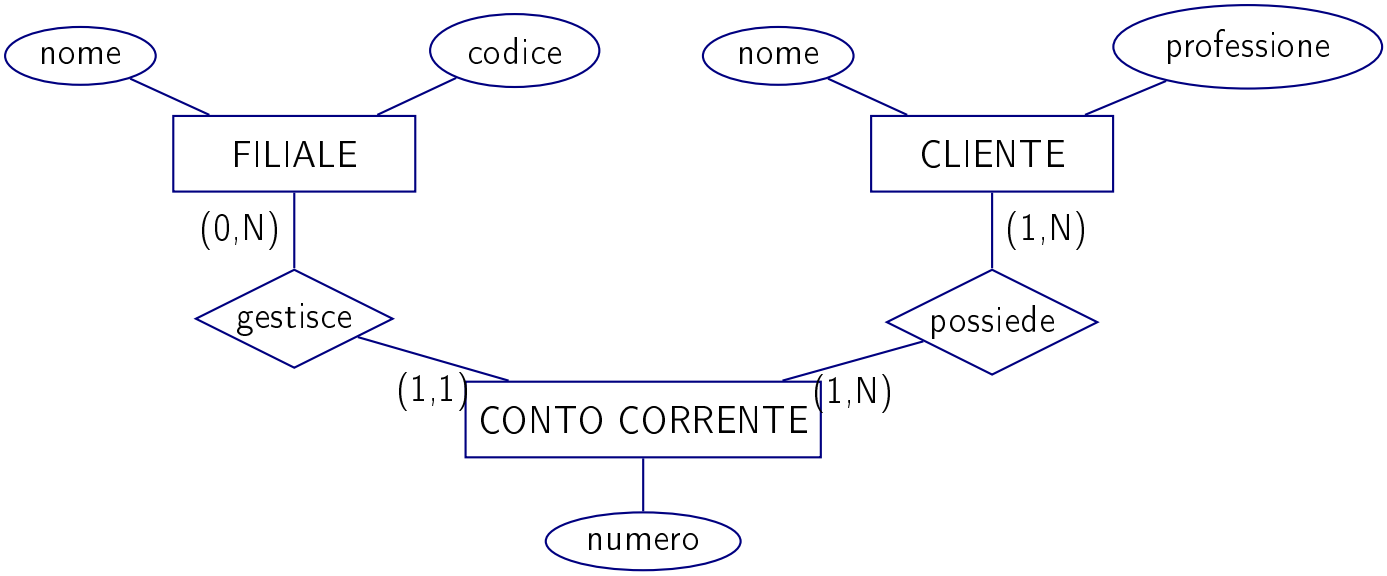


Figura 21: esercizio

filiali della banca. Ogni conto corrente viene aperto presso una e una sola filiale e ha una numerazione univoca nella filiale. Ogni cliente può aprire più conti (anche presso la stessa filiale) e un conto può avere più intestatari.
Si vuole mantenere traccia nella base di dati di tutte le operazioni (movimenti) eseguite sui conti correnti, indicando per ogni movimento, un numero progressivo univoco per conto corrente, la data, il tipo di operazione e il valore.
Per ogni conto corrente si deve produrre il saldo. Si supponga che un cliente possa aprire conti diversi in più filiali della banca.


nome
codice
nome
professione
FILIALE
CLIENTE
(0,N)
(1,N)
gestisce
possiede
(1,1)
(1,N)
CONTO CORRENTE
numero